NOVA IMS INFORMATION MANAGEMENT SCHOOL

Big Data & Cloud Computing

TP2 — Containers and execution environments

Prof. Lorenzo Brigato

MSc in Data Science and Advanced Analytics

Recall from TP1

- Compute executes instructions; persistent storage retains data.
- A virtual machine has a guest operating system.
- A container isolates processes and packages their software dependencies.
- A managed service takes over selected operating responsibilities.

Today: what must travel with a program so it can run reliably?

Today's lecture (TP2)

INTRODUCTION

- Describe a program's execution environment.
- Distinguish an image, a container, a host and a volume.
- Explain how an image is built and used.
- Trace inputs, saved outputs and failures.
- Identify the conditions needed for reproducible execution.

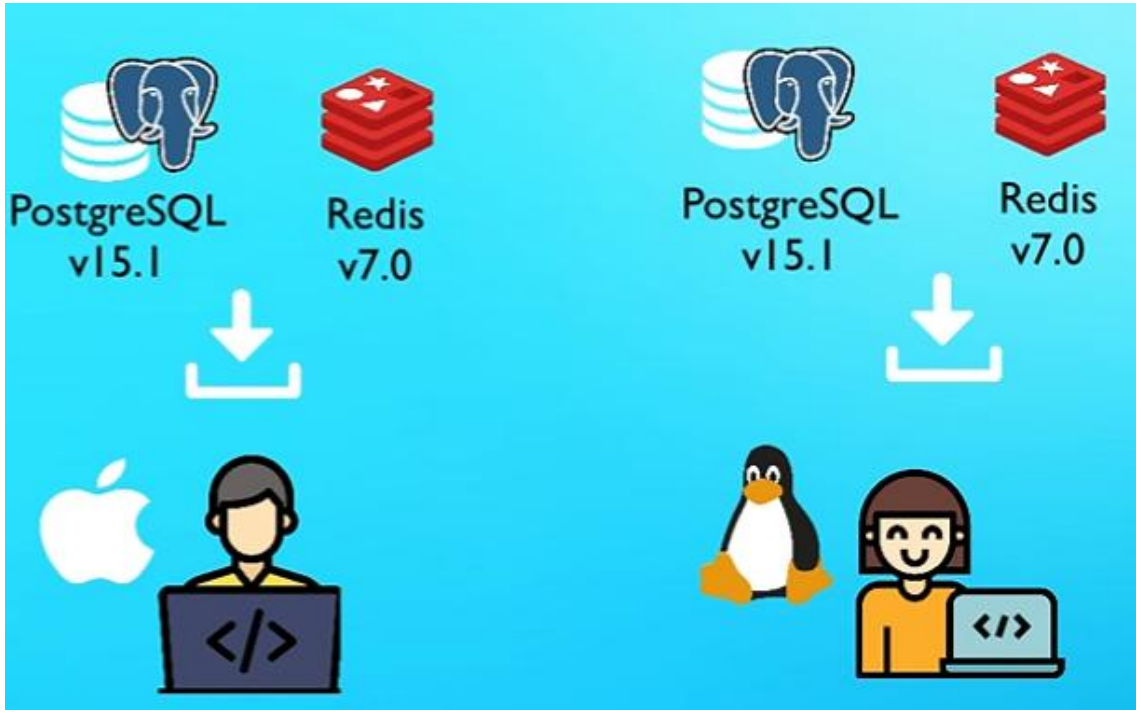
Focus on explaining what happens when software is packaged and run.

The portability problem

INTRODUCTION

The same source code is copied to three environments.

Environment	What happens?
Author’s laptop	Runs successfully
Colleague’s laptop	A required package is missing
Cloud server	The input path does not exist



Source code is only part of what a program needs.

The execution environment

EXECUTION ENVIRONMENTS

Part	Example
Application	The program and its version
Runtime and packages	Python interpreter and imported libraries
Operating system	Kernel, system libraries and permissions
Configuration and data	Parameters, input files and output locations
Resources	CPU, memory, storage and network access

A reliable run needs compatible software and suitable runtime conditions.

Program and process

EXECUTION ENVIRONMENTS

Program

Stored instructions, such as report.py

Process

An executing instance of those instructions

One program can be used by many processes.

Each run can use different data, configuration and resources.

Finishing a process and deleting its stored files are different actions.

Runtimes and dependencies

EXECUTION ENVIRONMENTS

Runtime

Software that executes the application, such as the Python interpreter

Dependency

Software the application needs: a package, a library or a system component

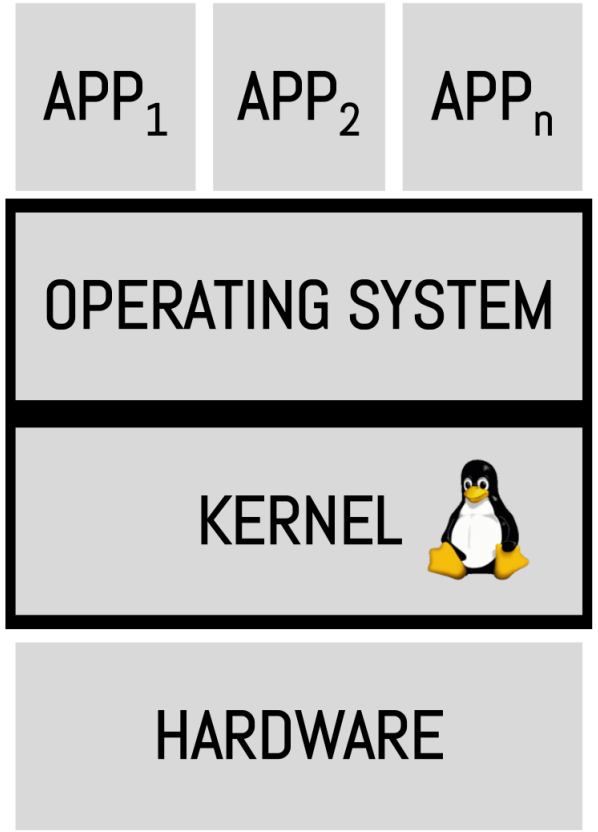
Dependencies can have dependencies of their own.

“Python is installed” does not fully specify an environment.

The operating-system kernel

The kernel is the core of an operating system.

- Schedules CPU work
- Manages memory
- Mediates access to devices and files
- Supports process isolation



Sharing a kernel is central to how ordinary Linux containers work.

Python environments and containers

EXECUTION ENVIRONMENTS

	Python virtual environment	Container
Main purpose	Separate Python packages	Package software and isolate processes
Runtime	Uses a Python installation	Can include a Python interpreter
System libraries	Still depend on the host	Can be included in the image
Kernel	Uses the host kernel	Shares a compatible host kernel

Choose the level of environment control the application needs.

Configuration, code and data

EXECUTION ENVIRONMENTS

Item	Example	Role
Code	validate.py	Defines the validation logic
Configuration	--input /input/transactions.csv	Selects the data location
Input data	The records for today	Provides the values to process
Output data	Accepted records and reasons	Records what the run produced

Configuration may come from arguments, environment variables or a file.

What should another person be able to repeat?

EXECUTION ENVIRONMENTS

- Use the intended application and dependency versions.
- Run it with the same parameters and the same input snapshot.
- Understand which outputs indicate success.
- Explain the conditions that can still change the result.

A repeatable environment is a foundation for a repeatable result.

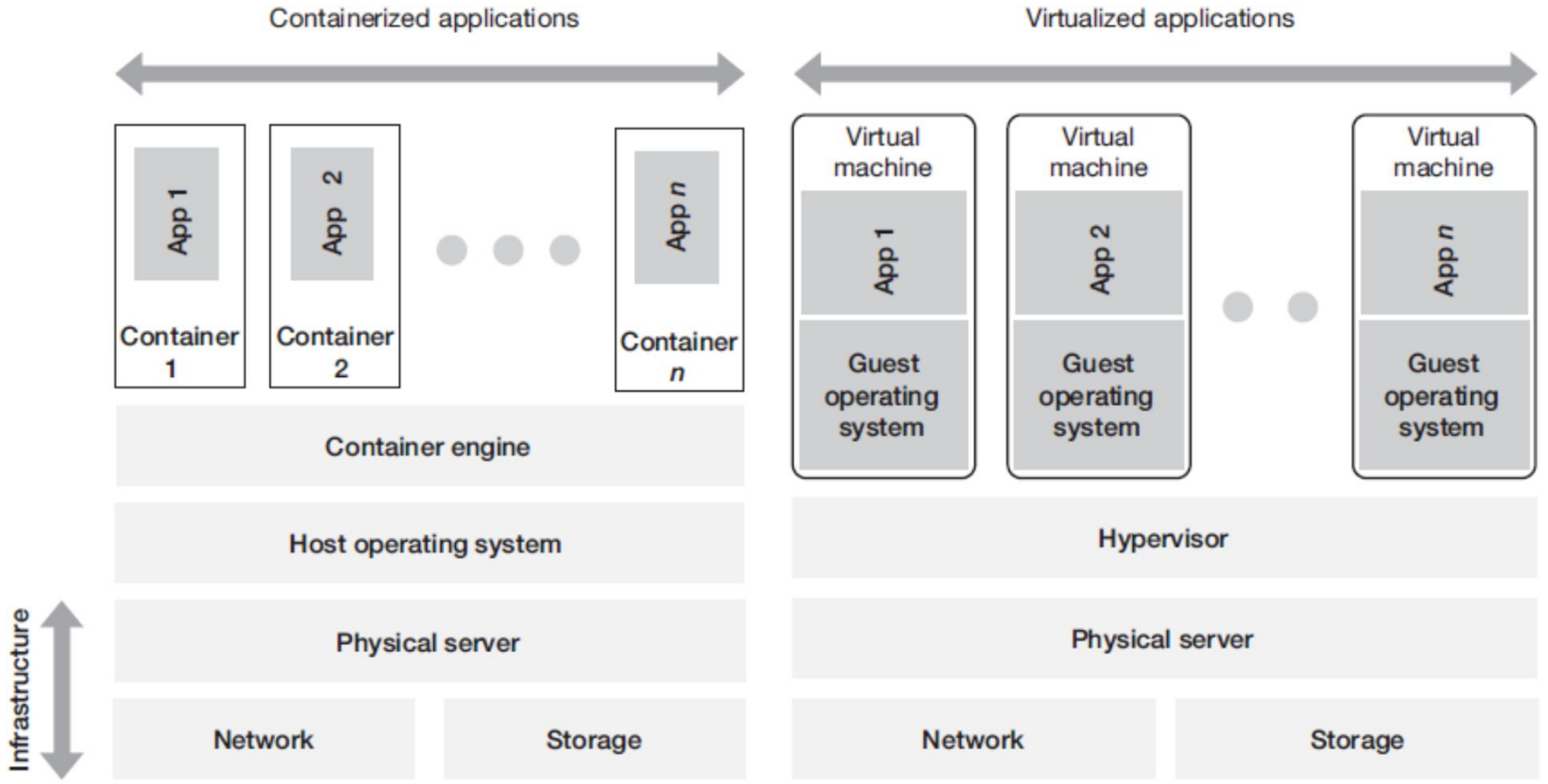
Containers and virtual machines

Virtual machine

Virtual hardware with a guest operating system and its own kernel

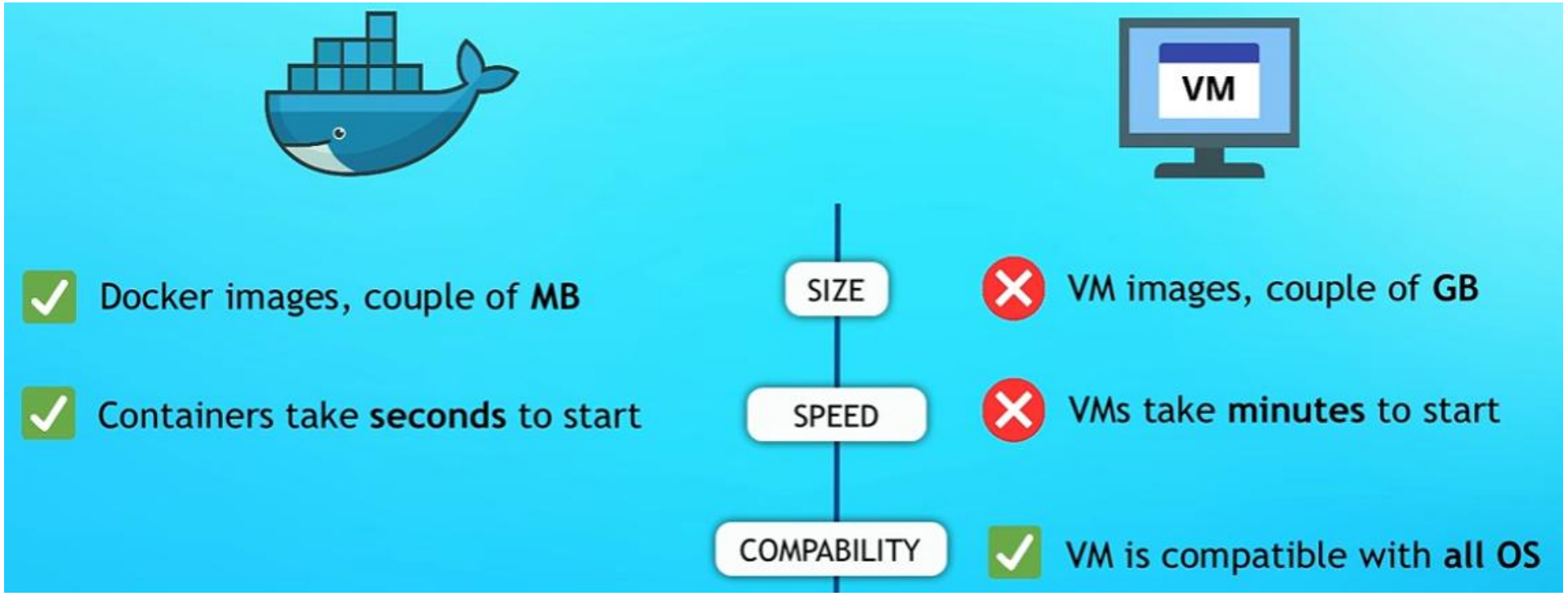
Container

Isolated processes with packaged dependencies, sharing a host kernel



Containers and virtual machines

Advantages and disadvantages considering size, speed, and compatibility

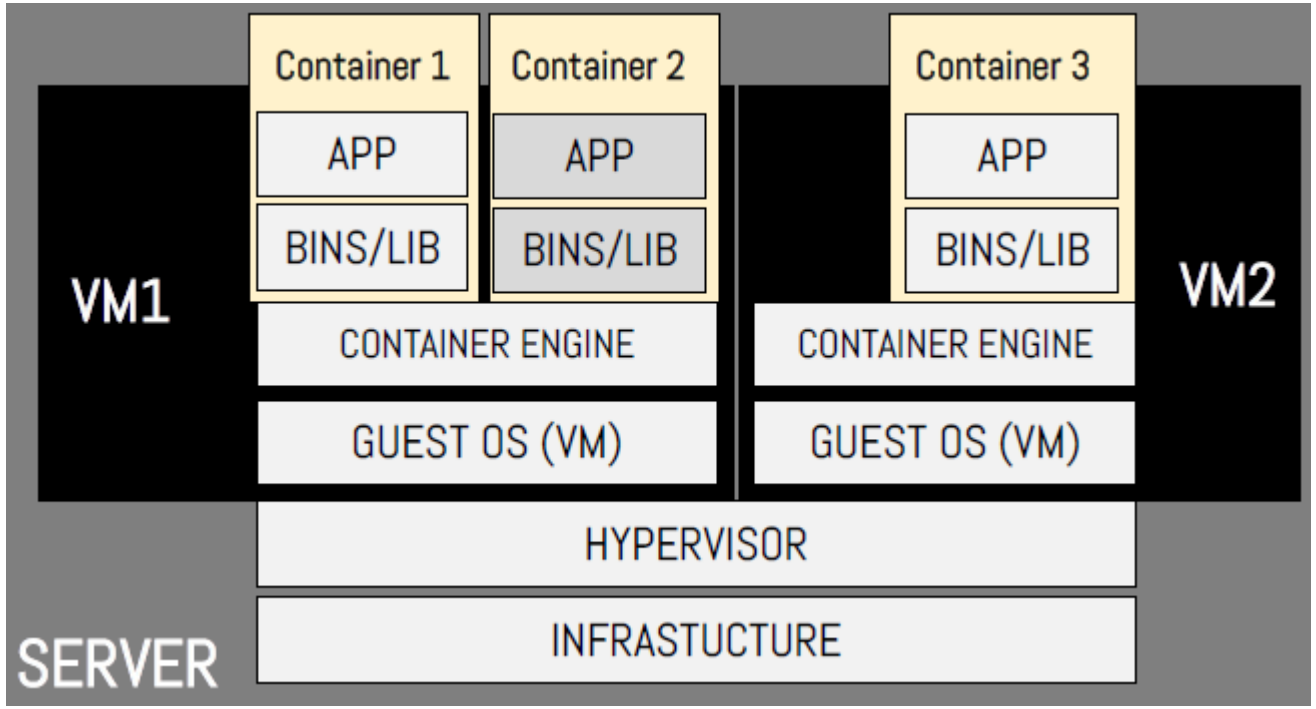


Containers can run inside VMs

The VM provides the operating-system environment.

Containers inside it share the VM's guest kernel.

For those containers, the VM is their host.



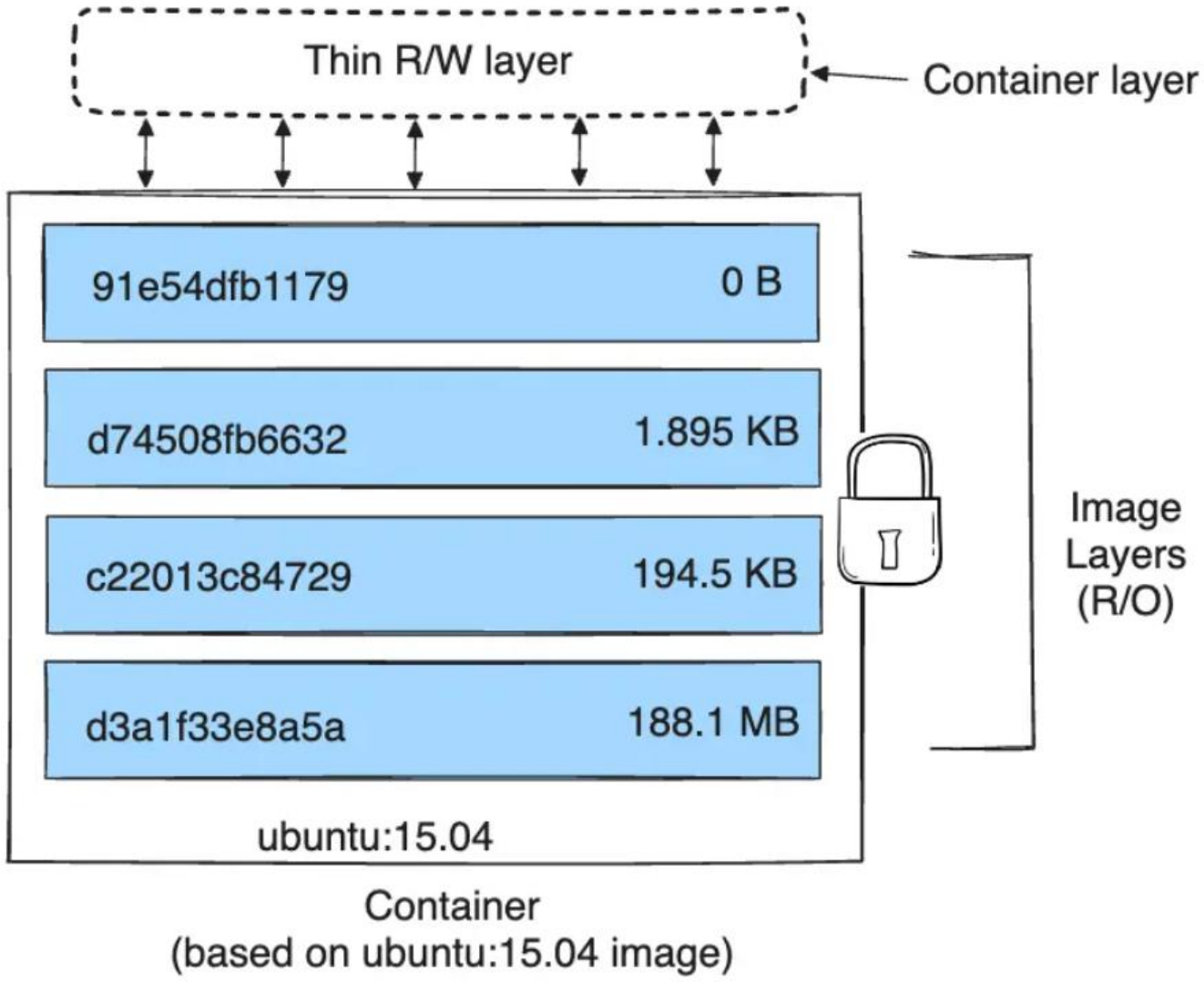
"Host" identifies the system running the containers.

Image and container

CONTAINERS AND VIRTUAL MACHINES

Image: read-only software package.

Container: an instance with its own writable layer.



Isolation and shared resources

A container has a controlled view of:

- Processes and the filesystem
- Network interfaces and permitted access
- Its own software dependencies

Containers still share the host kernel and physical resources.

Isolation depends on configuration; it is not an absolute security guarantee.

Portability has compatibility limits

CONTAINERS AND VIRTUAL MACHINES

- The operating-system kernel must be compatible.
- The image must support the CPU architecture.
- Required files, services and permissions must be available.
- Enough memory, CPU and storage must be provided.

Packaging removes many differences; it does not remove every dependency.

Packaging and resource allocation

A validator is moved into a Docker image.

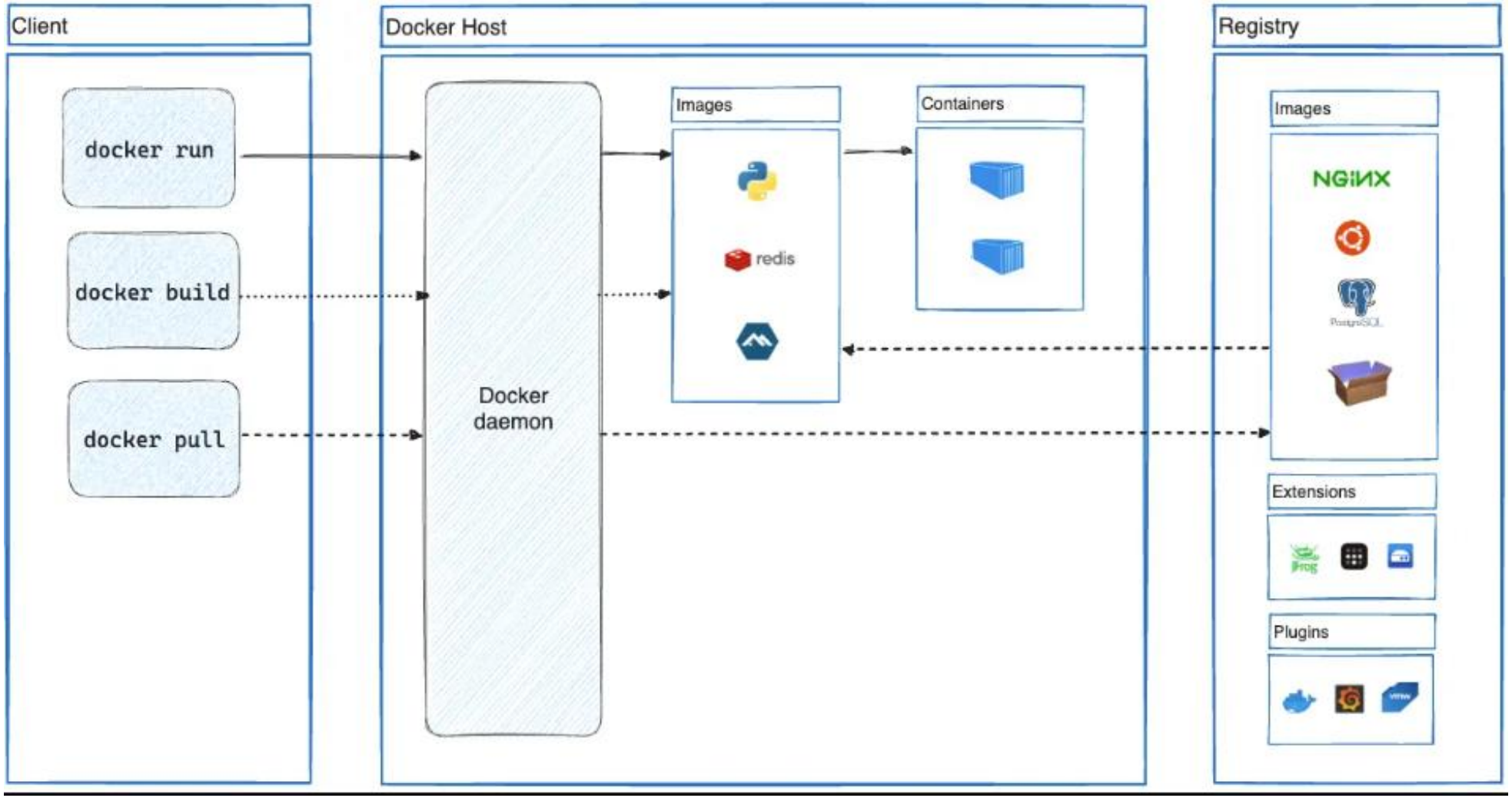
Does it now have more CPU?

No. Packaging and resource allocation are separate.

The host supplies the resources. Runtime settings can limit their use.

Docker components

IMAGES AND BUILDS



Build time and run time

IMAGES AND BUILDS

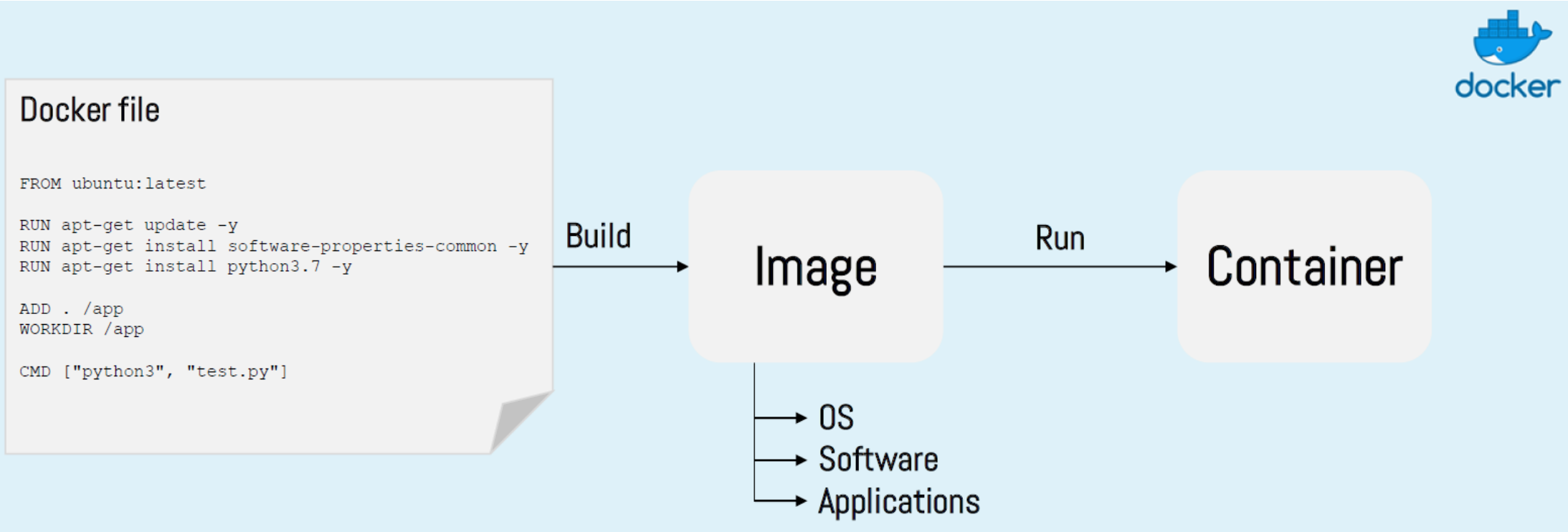
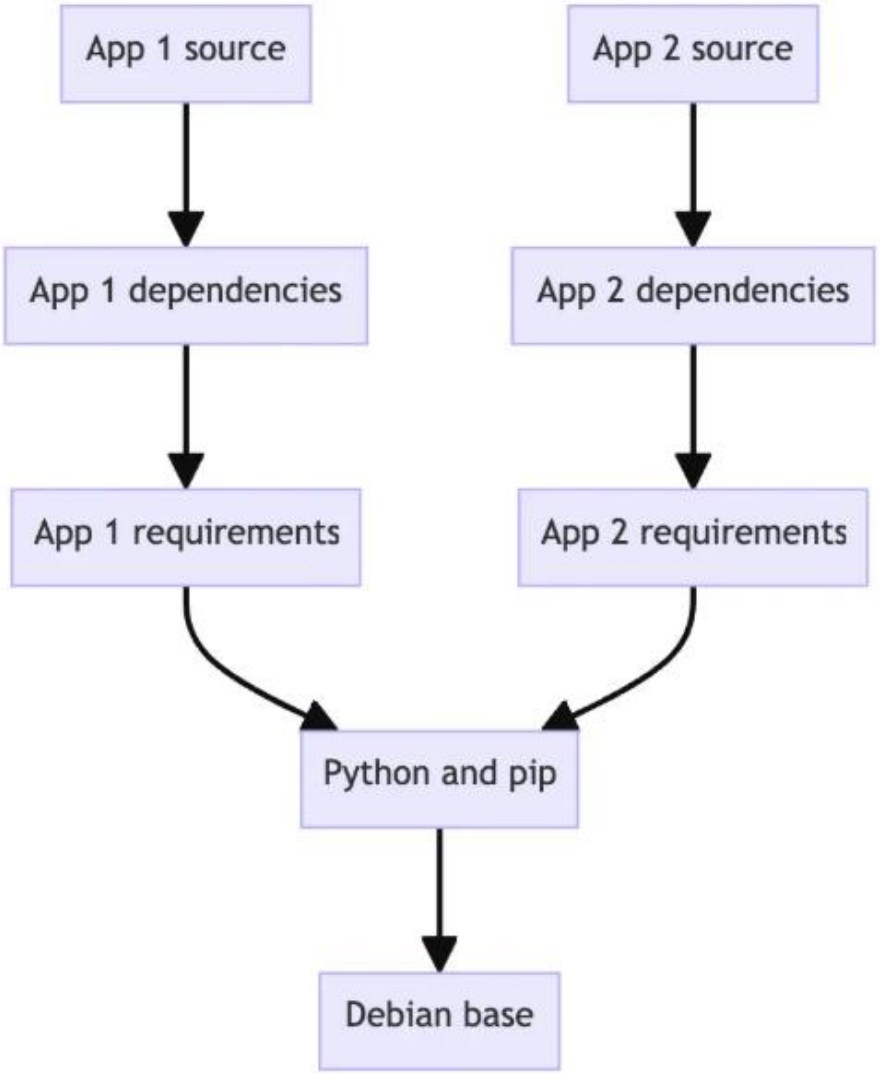


Image layers

An image combines reusable layers of filesystem content.

- Common base software can be shared.
- Application-specific layers add packages and code.
- The container sees one combined filesystem.



A Dockerfile for a validator

Teaching example · Python standard library only

```
FROM python:3.12.15-slim-bookworm
WORKDIR /app
COPY validate.py .
ENTRYPOINT ["python", "validate.py"]
CMD ["--input", "/input/transactions.csv", \
     "--output-dir", "/output"]
```

FROM selects the base.

WORKDIR and COPY place the program.

ENTRYPOINT and CMD define how a container starts.

Input records and results are supplied at run time.

RUN, ENTRYPOINT and CMD

Instruction	When it matters	Example role
RUN	During the image build	Install application dependencies
ENTRYPOINT	When the container starts	Select the validator executable
CMD	When the container starts	Provide default input/output arguments

Changing today's input path should not require rebuilding the software.

Sharing and identifying images

Registry → repository → image reference

Example: registry.example.edu/course/validator:1.0

- A registry distributes stored images.
- A repository groups images for an application.
- A tag names a version or variant chosen by its publisher.
- Pulling an image and running a container are separate actions.

Tags and digests

IMAGES AND BUILDS

Reference	Meaning	Can it change?
Tag, such as :1.0	A human-readable label	The label can point elsewhere
Digest, sha256:…	A content identifier	Changed content has a different digest

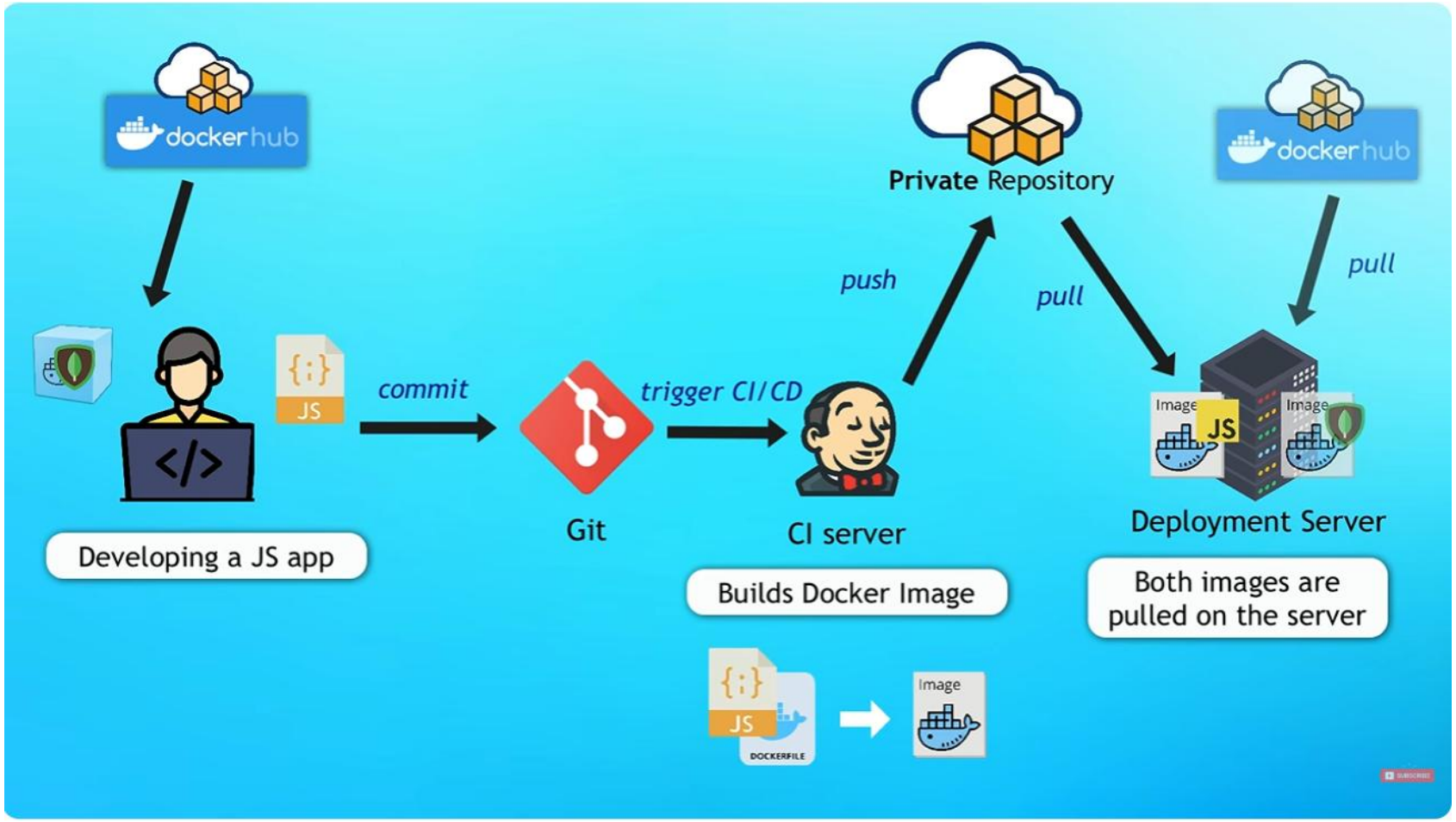
For a repeatable run, record the image digest and target platform.

Update deliberately when dependencies or security fixes change.

Docker in the software delivery process

IMAGES AND BUILDS

- **Develop:** write code and a Dockerfile.
- **Build and publish:** build the image and push it to a registry.
- **Deploy and run:** pull images and start containers.



A repeatable build needs recorded inputs

- Record the application code version.
- Identify the base image precisely.
- Record direct and transitive dependency versions.
- Limit the build context: files available to the image builder.
- Keep the resulting image reference and build record.

The same Dockerfile can produce a different image if its inputs change.

The container lifecycle

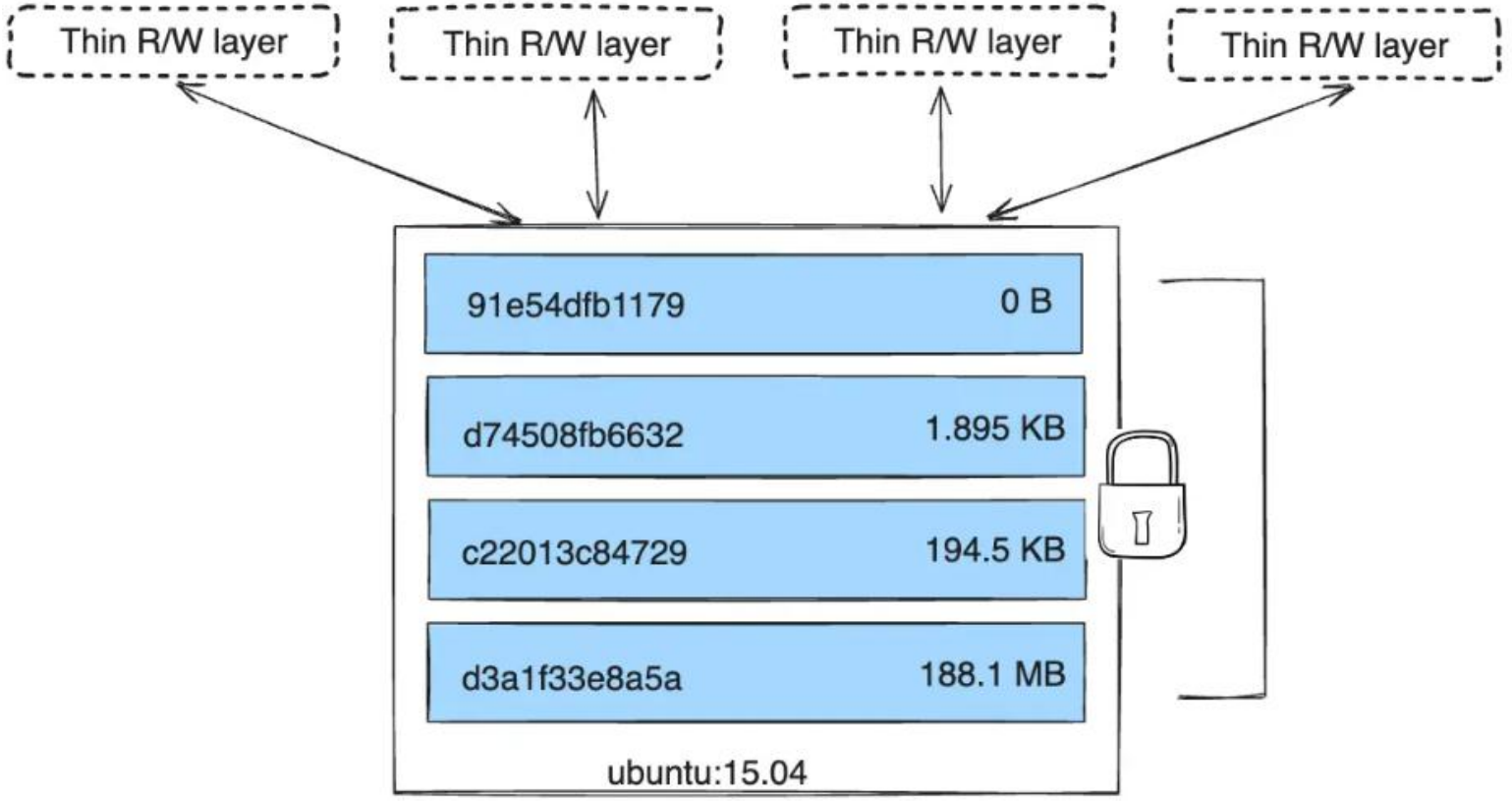
DATA AND CONTAINER LIFETIMES

State	What it means
Created	The container exists; its main process has not started
Running	Its processes are executing
Stopped	Its main process has exited; the container still exists
Removed	The container and its writable layer have been deleted

Stopped and removed are different states.

Where data live determines their lifetime

DATA AND CONTAINER LIFETIMES



Each container has its own writable layer.

Stopping retains it. Removing the container deletes it.

Mounted storage has a separate lifetime.

Save important results outside the container's writable layer.

Bind mounts and volumes

DATA AND CONTAINER LIFETIMES

Bind mount	Named volume
Links a chosen host path to a container path	Docker manages the storage location
Useful for accessible input and output folders	Useful for data reused across containers
Depends on the host path and permissions	Has an independent name and lifecycle

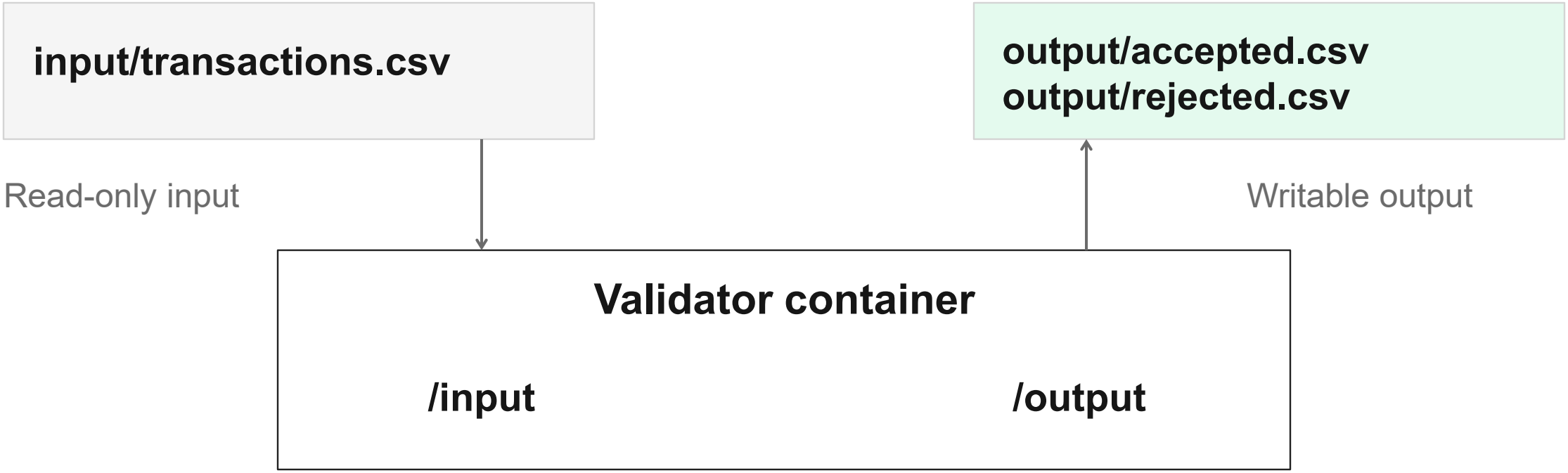
Inside the container, each appears as a file or directory.

The “host” is the system running Docker, which may be remote.

Input and output mounts

DATA AND CONTAINER LIFETIMES

External storage on the Docker host



The mount connects paths; it does not copy the files into the image.

A program's input/output contract

Input: /input/transactions.csv

transaction_id · transaction_timestamp (UTC)
customer_id · amount_cents · merchant

Checks Required values are present; timestamp is valid UTC; amount_cents is a positive integer.

Outputs accepted.csv: rows that satisfy the checks
rejected.csv: source row and reason for each rejection

Unavailable input or unwritable output: fail the run.

Validation and screening answer different questions

Validation

Can this record be interpreted under our data contract?

An invalid amount is a data-quality problem.

Screening

Does an accepted record meet the review rule?

amount_cents > 100000

A valid transaction can be flagged. A flag is not proof of fraud.

Worked validation example

INPUTS, OUTPUTS AND FAILURES

Synthetic records · other required fields are valid

ID	amount_cents	customer_id	Validation result
T1	10000	C01	Accepted
T2	150000	C02	Accepted
T3	100000	C03	Accepted
T6	−500	C06	Rejected: non-positive amount
T7	4250	(missing)	Rejected: missing customer

Accepted: 3 Rejected: 2 Flagged after screening: T2 only

Bad records and failed runs

INPUTS, OUTPUTS AND FAILURES

Situation	Outcome in this example	Exit status
Two invalid records	Write accepted and rejected outputs	0
Input file is missing	Write an error; no new result files	2
Output cannot be written	Fail; outputs are not complete	Non-zero

A rejected record and an unsuccessful execution are different events.

The program defines its policy and documents its exit codes.

Logs, outputs and exit status

INPUTS, OUTPUTS AND FAILURES

Signal	Purpose	Illustrative value
Saved output	The data produced	accepted.csv
Log message	Explanation of execution	accepted=3 rejected=2
Error message	What went wrong	input file not found
Exit status	Success/failure signal	0 or a documented non-zero code

Read the status and the explanation; a file's presence alone is insufficient.

Trace a successful run and a failure

	Run A	Run B
Input	Five records shown earlier	Missing file
External output folder	output/run-A/	output/run-B/ (fresh)
Saved result	3 accepted; 2 rejected	No new result files
Exit status	0	2
After container removal	Run A files still available	No result to retain

External output survives the container; failure still needs an explicit signal.

The record of a run

REPRODUCIBILITY AND RECAP

- Application version, image digest and target platform
- Input snapshot or file checksum
- Parameters and relevant environment settings
- Output location, row counts and completion status
- Host resources, start time and elapsed time

Record what ran, what it read and how completion was established.

Reproducibility limits

What is repeated?	What else must be controlled?
Software environment	Image identity, platform compatibility
Analytical result	Inputs, parameters and non-deterministic behaviour
Execution time	Hardware, resource limits, load, I/O and caches

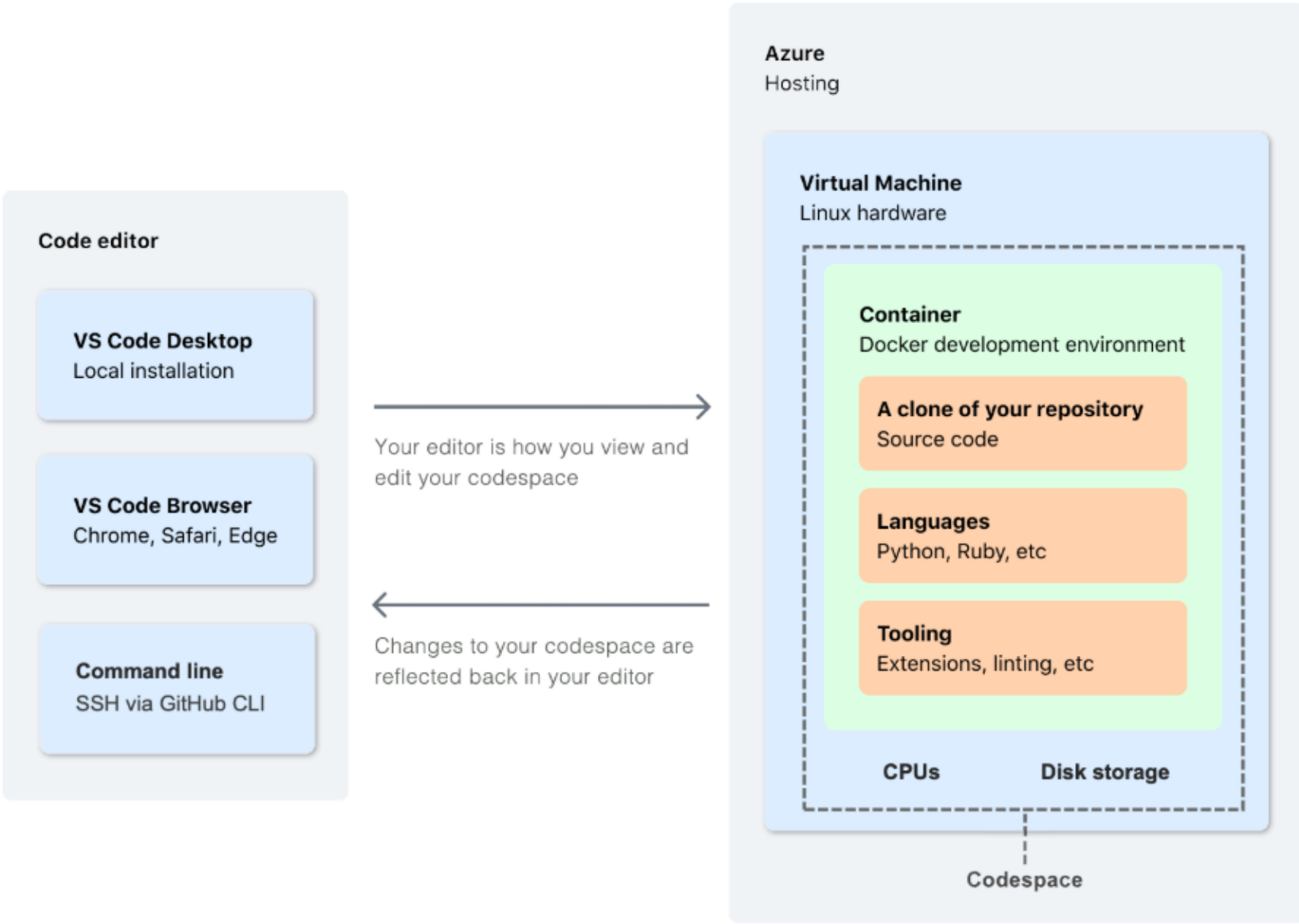
An unchanged image does not guarantee identical performance.

Specify which result should agree and what differences are acceptable.

The course execution environment

REPRODUCIBILITY AND RECAP

A **Codespace** is a remote **development environment** hosted by GitHub **in a container on a VM**.



Concept check

REPRODUCIBILITY AND RECAP

Does deleting a container necessarily delete mounted output?

No. External storage has a separate lifecycle.

Does an image provide extra CPUs?

No. Resource allocation is separate.

Does using the same tag prove the image content is identical?

No. A tag can point to different content.

Do rejected records necessarily mean the run failed?

No. That depends on the application’s contract.

Main ideas and reading

- Package the intended software environment.
- Distinguish images from their container instances.
- Make configuration and input/output behaviour explicit.
- Retain important data outside the container’s writable layer.

Reading

Ruparelia: Ch. 1, pp. 9–12; Ch. 4, pp. 61–66
Reis & Housley: Ch. 4, pp. 144–146

Next: Computing performance and scaling